

HW 6 — 1D Lagrangian Hydrodynamics: the Sod Shock Tube

Eran Rehani, 207823063

This document explains the assignment, the algorithms behind the notebook `HW_06.ipynb`, and how the code implements them. For each part: the theory first, then the implementation.

1. What the assignment asks

A 1D ideal gas, $p = (\gamma - 1)\rho e$ with $\gamma = 1.4$, fills a tube $x \in [0, 16]$ (cgs units). At $t = 0$ there is a diaphragm at $x = 8$ separating two uniform states at rest:

$$(\rho, u, p) = \begin{cases} (1, 0, 1) & x < 8 \\ (0.125, 0, 0.1) & x > 8 \end{cases}$$

Both ends are rigid walls (acceleration = 0 there). The task:

1. Evolve the system to $t = 2.5$ with the Lagrangian difference equations of summary 19, including artificial viscosity.
2. Do it on four grids: $N = 34, 100, 334, 1000$ intervals.
3. For each grid, plot the density profile $\rho(x)$ at the final time.
4. For each grid, tabulate the total-energy non-conservation fraction $(E_{\text{tot}}^{\text{end}} - E_{\text{tot}}^{\text{start}})/E_{\text{tot}}^{\text{start}}$ and the propagation speeds of the four flow features: the shock, the contact surface, and the two edges of the rarefaction fan. The speeds are extracted from two density profiles, at $t = 2.0$ and $t = 2.2$.

This initial condition is the classic **Sod shock tube** — the standard benchmark for compressible-flow codes, because it has an exact solution to compare against.

2. The physics: what comes out of the jump

When the diaphragm “breaks”, the pressure difference pushes the dense left gas into the light right gas. Three waves emerge, all starting at $x = 8$:

- **Shock** (moves right, fast): a jump in p , ρ and u together. The undisturbed right-hand gas is compressed and set into motion abruptly. A shock is a genuinely dissipative feature — entropy increases across it.
- **Contact surface** (moves right, slower): the material boundary between gas that was originally on the left and gas originally on the right. Density jumps across it but pressure and velocity are continuous — nothing pushes on it, it just gets carried along with the flow ($v_{\text{contact}} = u$ there).
- **Rarefaction fan** (moves left): the left gas expands smoothly into the evacuated region. It is a continuous wave with two edges: the **head** (the first signal, moving into the undisturbed left state at the sound speed, $u_L - c_L$) and the **tail** (its trailing edge, at $u_* - c_{*L}$).

Between the tail and the shock sits the “star” region: constant pressure p_* and velocity u_* , with the contact in the middle separating two different densities $\rho_{*L} \neq \rho_{*R}$.

The whole flow depends only on the similarity variable $\xi = (x - 8)/t$ (there is no length scale in the problem until a wave hits a wall), which is also why each feature moves at a constant speed — that’s what makes the speed measurement in part 4 well-defined.

3. Lagrangian description and the staggered grid

Theory

In a Lagrangian scheme the grid moves **with** the gas: each computational cell always contains the same gas, so its mass m is constant in time and the advection terms disappear from the equations. What changes is the cell's volume (here, length) — compression shows up directly as the cell shrinking. This is a natural fit for 1D flows with sharp features: cells crowd into the shock by themselves.

The grid is **staggered**:

- **Vertices** $i = 0, \dots, N$ carry position x_i , velocity u_i , acceleration a_i .
- **Cells** $i - \frac{1}{2}$ (between vertices $i - 1$ and i) carry ρ , e , p , q , and the fixed mass $m_{i-1/2}$.

The cell volume is $V_{i-1/2} = x_i - x_{i-1}$ and the density is just $\rho_{i-1/2} = m_{i-1/2}/V_{i-1/2}$ — mass conservation is exact by construction, to machine precision. The staggering puts every derivative exactly where it is needed: the pressure gradient (a difference of cell values) lives on a vertex and accelerates it; the velocity divergence (a difference of vertex values) lives in a cell and compresses it. That centering is what makes the scheme second order in space.

Implementation (init_state)

`init_state(N)` builds $N+1$ equally spaced vertices on $[0, 16]$. All four values of N are even, so the diaphragm $x = 8$ falls exactly on a vertex and every cell is unambiguously “left” or “right”. Cell centres pick ρ and p from the proper side, $e = p/((\gamma - 1)\rho)$, and $m = \rho \Delta x$ is stored once and never modified. Velocities and accelerations start at zero.

4. The von Neumann-Richtmyer scheme

Theory

One time step of size Δt runs equations (1)-(8) of summary 19 in order. Velocities live on half-integer time levels (leapfrog), which centers all the time derivatives:

$$\begin{aligned} u_i^{n+1/2} &= u_i^n + \frac{\Delta t}{2} a_i^n, & x_i^{n+1} &= x_i^n + \Delta t u_i^{n+1/2}, \\ V_{i-1/2}^{n+1} &= x_i^{n+1} - x_{i-1}^{n+1}, & \rho_{i-1/2}^{n+1} &= m_{i-1/2}/V_{i-1/2}^{n+1}. \end{aligned}$$

The internal energy follows the first law, $de = -p d(1/\rho)$ per unit mass, discretized with the time-averaged pressure so the work term is centered:

$$e^{n+1} = e^n - \frac{1}{2m} [(p^{n+1} + p^n) + (q^{n+1} + q^n)] (V^{n+1} - V^n).$$

Then the new acceleration from the momentum equation, and the second velocity half-step:

$$a_i^{n+1} = -2 \frac{(p + q)_{i+1/2}^{n+1} - (p + q)_{i-1/2}^{n+1}}{m_{i-1/2} + m_{i+1/2}}, \quad u_i^{n+1} = u_i^{n+1/2} + \frac{\Delta t}{2} a_i^{n+1}.$$

The walls enforce $a_0 = a_N = 0$ (and $u = 0$ there, since the gas starts at rest).

Artificial viscosity

Without dissipation, the difference equations are adiabatic — they conserve entropy. But a real shock *creates* entropy, so an adiabatic scheme cannot produce one; instead it develops unphysical oscillations behind the steep front. The von Neumann-Richtmyer fix is an artificial viscous pressure that turns on only in compression:

$$q_{i-1/2}^{n+1} = \begin{cases} \sigma \rho_{i-1/2}^{n+1} (u_{i-1}^{n+1/2} - u_i^{n+1/2})^2, & u_i^{n+1/2} < u_{i-1}^{n+1/2} \\ 0, & \text{otherwise} \end{cases}$$

with $\sigma = 3$ here. Since q acts like a pressure, it is added to p everywhere p appears (the energy equation and the acceleration). Because $q \propto (\Delta u)^2$, it is significant only where the velocity changes a lot over one cell — i.e. inside the shock front — and negligible in smooth regions, so the rarefaction fan is essentially untouched. The effect is to smear the shock over a few cells (roughly $\sqrt{\sigma}$ cells wide) while producing the correct entropy jump, so the shock has the right strength and the right speed.

The implicit energy-pressure coupling

The energy equation above contains p^{n+1} , which itself depends on e^{n+1} through the equation of state — formally an implicit equation. For an ideal gas it can be solved exactly instead of iterated: substituting $p^{n+1} = (\gamma - 1)(m/V^{n+1})e^{n+1}$ and writing $\Delta V = V^{n+1} - V^n$, everything with e^{n+1} moves to the left side:

$$e^{n+1} \left[1 + \frac{(\gamma - 1) \Delta V}{2 V^{n+1}} \right] = e^n - \frac{\Delta V}{2m} (p^n + q^n + q^{n+1})$$

$$\Rightarrow e^{n+1} = \frac{e^n - \frac{\Delta V}{2m} (p^n + q^n + q^{n+1})}{1 + \frac{(\gamma - 1) \Delta V}{2 V^{n+1}}}, \quad p^{n+1} = (\gamma - 1) \rho^{n+1} e^{n+1}.$$

(q^{n+1} is already known at this point — it only needs ρ^{n+1} and $u^{n+1/2}$.)

Implementation (step)

`step(s, dt)` is a direct transcription, fully vectorized (the only Python loop in the whole simulation is over time steps):

1. `u_half = u + 0.5*dt*a`, walls pinned to zero.
2. `x_new = x + dt*u_half`; `V_new = np.diff(x_new)`; `rho_new = m/V_new`.
3. `du = u_half[:-1] - u_half[1:]` is $u_{i-1} - u_i$ per cell; `q_new = np.where(du > 0, SIGMA*rho_new*du**2, 0)` — positive `du` means the cell's ends are approaching, i.e. compression.
4. The closed-form `e_new`, then `p_new = (GAMMA-1)*rho_new*e_new`.
5. `a_new[1:-1] = -2*(pq[1:]-pq[:-1])/(m[:-1]+m[1:])` with `pq = p_new + q_new`; the wall entries stay 0.
6. `u_new = u_half + 0.5*dt*a_new`.

5. Time-step control

Theory

Being explicit, the scheme is stable only under a Courant condition — information (sound) must not cross a whole cell in one step:

$$\Delta t \leq \text{CFL} \cdot \min_i \frac{\Delta x_i}{c_i}, \quad c = \sqrt{\gamma p / \rho},$$

with a safety factor $\text{CFL} = 0.2$. On top of that, accuracy requires that no cell's density change by more than a few percent in one step. Since $\dot{\rho}/\rho = -\dot{V}/V = -(u_i - u_{i-1})/V$, this gives a second bound:

$$\Delta t \leq \text{MAX_DRHO} \cdot \min_i \frac{\Delta x_i}{|u_i - u_{i-1}|}, \quad \text{MAX_DRHO} = 0.05.$$

Implementation (`compute_dt`)

Takes the minimum of the two bounds (the density bound only over cells that are actually moving, to avoid division by zero at $t = 0$ when $u \equiv 0$), never lets Δt grow by more than 10% between consecutive steps (so the step size changes smoothly), and clamps the last step so the run lands exactly on each snapshot time ($t = 2.0, 2.2, 2.5$) instead of stepping past it.

6. Total energy and the non-conservation fraction

Theory

The assignment defines the total energy as internal plus kinetic, with each piece summed where it naturally lives on the staggered grid:

$$E_{\text{tot}} = \underbrace{\sum_{\text{cells}} e_{i-1/2} m_{i-1/2}}_{\text{internal}} + \underbrace{\frac{1}{2} \sum_{\text{vertices}} M_i u_i^2}_{\text{kinetic}}, \quad M_i = \frac{1}{2} (m_{i-1/2} + m_{i+1/2}),$$

where the vertex mass M_i is half of each neighbouring cell and the wall vertices own half of their single neighbour cell (so $\sum M_i = \sum m_{i-1/2}$ — no mass is counted twice).

The scheme conserves mass and momentum exactly, but **not** total energy: the energy equation updates e from the $p dV$ work, and the discrete kinetic energy bookkeeping doesn't cancel the discrete work term exactly (the scheme is not written in conservation form). The residual $(E^{\text{end}} - E^{\text{start}})/E^{\text{start}}$ is therefore a quality measure of the calculation — it should be small and shrink as the grid is refined.

Implementation (`total_energy`, `run`)

`total_energy` is three lines of `np.sum`. `run(N, snap_times)` records E_{start} before the first step, marches through the snapshot times saving (x_c, ρ, p, u) at each, asserts $\rho, p > 0$ throughout, and returns the snapshots together with the energy fraction.

7. The exact Riemann solution (the reference)

Theory

The Sod problem has an exact solution, which the notebook uses both as the overlay on the density plots and as the reference for the measured speeds. The star pressure p_* satisfies

$$f(p_*) = f_L(p_*) + f_R(p_*) + (u_R - u_L) = 0,$$

where each side contributes a shock branch (from the Rankine-Hugoniot conditions) if $p_* > p_K$, or a rarefaction branch (from the isentrope + Riemann invariant) if $p_* < p_K$:

$$f_K(p) = \begin{cases} (p - p_K) \sqrt{\frac{A_K}{p + B_K}}, & p > p_K \\ \frac{2c_K}{\gamma - 1} \left[\left(\frac{p}{p_K} \right)^{\frac{\gamma-1}{2\gamma}} - 1 \right], & p \leq p_K. \end{cases}$$

where $A_K = \frac{2}{(\gamma + 1)\rho_K}$, $B_K = \frac{\gamma - 1}{\gamma + 1}p_K$.

$f(p)$ is monotonic, so the root is unique; from p_* follow u_* , the two star densities (isentropic on the left, Hugoniot on the right), and the four wave speeds. For this data:

$$p_* \approx 0.3031, \quad u_* \approx 0.9275, \quad v_{\text{head}} \approx -1.1832, \quad v_{\text{tail}} \approx -0.0702, \quad v_{\text{contact}} \approx 0.9275, \quad v_{\text{shock}} \approx 1.7522.$$

These are the standard published Sod values, which is itself a check on the solver.

Implementation (sod_exact)

`_f_branch` implements f_K ; the root is found with `scipy.optimize.brentq` on $[10^{-8}, 10]$ (bisection-safe since f is monotonic). `sod_exact()` returns the star state, the four speeds, and a `profile(x, t)` function that samples the exact (ρ, u, p) by classifying each point's $\xi = (x - 8)/t$ into the five regions (undisturbed left / fan / star-left / star-right / undisturbed right), with the standard self-similar expressions inside the fan.

8. Measuring the feature speeds from the simulation

Theory

Per the assignment, each speed is obtained from two density profiles:

$$v_{\text{feature}} = \frac{x_{\text{feature}}(2.2) - x_{\text{feature}}(2.0)}{0.2}.$$

The grid spacing on the coarse runs is large ($\Delta x = 0.47$ at $N = 34$), so just picking the nearest cell would quantize positions to Δx and ruin the speed estimate (a one-cell error is $0.47/0.2 \approx 2.4$ in speed units!). Instead, each position is defined as a **level crossing**: the (linearly interpolated) x where the profile crosses a fixed reference value. That pins each feature to sub-cell accuracy, and using the *same* level at both times makes the displacement consistent.

The choice of level per feature, using the exact star values purely as fixed reference levels:

- **Shock** — found on the *pressure* profile, at the half-jump $p = \frac{1}{2}(p_* + p_R)$, scanning from the right. Pressure is used rather than density because density jumps at the contact too, while pressure jumps only at the shock — this disambiguates the two unmistakably.
- **Contact** — half of the density jump, $\rho = \frac{1}{2}(\rho_{*L} + \rho_{*R})$, searched between the fan and the shock position.
- **Fan head and tail** — the fan is self-similar, so any fixed density value inside it moves at a strictly constant speed; the head is tracked at $\rho = 0.995 \rho_L$ (just departing the left plateau) and the tail at $\rho = 1.005 \rho_{*L}$ (just arriving at the star plateau).

A caveat the results table makes visible: the **tail barely moves** ($v \approx -0.07$, i.e. a displacement of 0.014 between the two snapshots — a thirtieth of a coarse-grid cell). On $N = 34$ and $N = 100$ its measured speed is meaningless noise; it only becomes a real measurement on the finer grids. The other three features move by one cell or more even at $N = 34$.

Implementation (`_cross`, `feature_positions`, `measure_speeds`)

`_cross(x, y, level, lo, hi, from_right)` walks the profile inside a window and returns the linear interpolation of the first bracketing pair. `feature_positions` applies the four rules above to one snapshot; `measure_speeds` differences the $t = 2.0$ and $t = 2.2$ positions.

9. Results

Energy non-conservation (also plotted in the notebook, log-log):

N	$(E^{\text{end}} - E^{\text{start}})/E^{\text{start}}$
34	$+6.4 \times 10^{-5}$
100	$+2.1 \times 10^{-5}$
334	$+6.3 \times 10^{-6}$
1000	$+2.1 \times 10^{-6}$

It decreases monotonically, with a log-log slope ≈ -1 , i.e. the energy error $\sim 1/N$ — consistent with the error being dominated by the discontinuities (locally first-order features), even though the scheme is second order in smooth flow.

Measured speeds vs. exact (relative error):

N	shock	contact	head	tail
34	2.4%	12.8%	9.2%	(noise)
100	2.0%	1.5%	3.2%	(noise)
334	0.6%	0.1%	0.1%	7.6%
1000	0.05%	0.01%	0.6%	5.1%

At $N = 1000$ the shock speed is 1.7530 vs. the exact 1.7522 — agreement to 5×10^{-4} . The tail behaves exactly as the resolution argument in §8 predicts.

Density profiles. All four grids reproduce the full Sod structure. The fan, being smooth, is well captured even at $N = 34$; the shock and contact are smeared over a few cells by the artificial viscosity and sharpen steadily with N , converging visibly onto the exact profile.

Validity check. The fastest wave (the shock) reaches only $x \approx 8 + 1.75 \times 2.5 = 12.4 < 16$ by the final time, so neither wall is ever touched by the flow — comparing against the exact *open-domain* Riemann solution is legitimate, and the rigid-wall boundary condition never actually does anything beyond holding the edge vertices in place.

10. Summary

The von Neumann-Richtmyer scheme with $\sigma = 3$ artificial viscosity reproduces all four features of the Sod problem at the correct speeds, converges to the exact solution as the grid is refined, and conserves total energy to a few parts in 10^5 even on the coarsest grid. The two known limitations are inherent and visible in the results: discontinuities are smeared over a few cells (the price of the viscosity), and features that move much less than a cell between the measurement times (the fan tail on coarse grids) cannot be assigned a meaningful speed.